

SUMMARY

This video explains the Minimal Deterministic Finite Automaton (MDFA) construction methodologies using heuristic “AD Rules.” These rules allow direct computation of minimal state counts without full diagram construction.

Key insights:

Precise formulas determine minimal state counts.

Left-end positional constraints grow linearly.

Right-end positional constraints grow exponentially.

Suffix constraints require $n + 1$ states.

Modulo length constraints require exactly n states.

I. MINIMAL DFA FOR STRING LENGTH CONSTRAINTS

When languages are defined strictly by string length over alphabet $\Sigma = \{a, b\}$, the following rules apply.

I.1 State Count Rules

Constraint Type	Length Condition	Minimal States
Exactly n	$\text{Length} = n$	$n + 2$
At Most n	$\text{Length} \leq n$	$n + 2$
At Least n	$\text{Length} \geq n$	$n + 1$

I.2 Construction Logic

Exactly n

Count transitions from 0 to n .

State at count n is final.

Any additional input transitions to a dead state.

At Most n

States representing counts 0 through n are final.

Exceeding n transitions to a dead state.

At Least n

Count until n .
Once reached, move to a final state with self-loop.
No dead state required.

II. DIVISIBILITY AND MODULAR ARITHMETIC IN DFAs

Languages defined by length divisibility follow cyclic state construction.

II.1 Length Divisible by n

Condition: $\text{Length} \equiv 0 \pmod{n}$

Property	Value
State Count	n
Final State	q_0
Structure	Cyclic transitions returning to q_0 after n symbols

II.2 $\text{Length} \equiv r \pmod{n}$

Rule:

Number of states = n

Each state represents remainder 0 through $n - 1$.
State corresponding to remainder r is final.

Example

$\text{Length} \equiv 2 \pmod{4}$

States = 4

Final State = q_2

III. SUBSTRING SUFFIX PATTERNS ("ENDS WITH")

For languages defined by ending with substring s of length n :

Minimal States = $n + 1$

III.1 Examples

Suffix	Length n	States (n + 1)
Ends with 0	1	2
Ends with 10	2	3
Ends with 011	3	4

III.2 Construction Logic

Build linear progression toward suffix match.

Add fallback transitions to preserve determinism.

Add self-loops where necessary to handle overlapping suffix patterns.

IV. POSITIONAL CONSTRAINTS FROM THE RIGHT END

Right-end positional constraints require tracking the last n symbols, resulting in exponential growth.

IV.1 Exponential State Growth Rule

If the n-th symbol from the right is fixed:

Minimal States = 2^n

Final States = $2^{(n-1)}$

Constraint	Minimal States
2nd symbol from right	4
3rd symbol from right	8
5th symbol from right	32
10th symbol from right	1024

Reason

The DFA must remember all possible combinations of the last n symbols.

IV.2 AD Table-Based Construction Method

Step 1: Identify the fixed symbol.

Step 2: Under the fixed-symbol column, assign odd-numbered states.

Step 3: Under the other-symbol column, assign even-numbered states.
Step 4: Repeat pattern for all 2^n states.

This structured mapping eliminates the need for full diagram drawing.

V. CASE STUDIES AND EXAM APPLICATIONS

V.1 Intersection of Languages

Machine M: Strings ending in a

Machine N: Strings ending in b

Result:

$$L(M) \cap L(N) = \emptyset$$

DFA for empty language:

State s	Description
1	Non-final state with self-loops

V.2 Regular Expression to DFA

Expression:

$$(0 + 1)^* 10$$

Meaning:

All strings ending with 10

Suffix length $n = 2$

$$\text{Minimal States} = 2 + 1 = 3$$

V.3 Comparative Complexity

Constraint Type	States Required
10th symbol from left	11
10th symbol from right	1024

Left-end constraints require linear state growth.
Right-end constraints require exponential memory representation.

